

17

Introducing React Server Components

After implementing our static blog app in Next.js, it's time to introduce some interactivity to it. Instead of using the traditional pattern of writing a separate backend server, which the frontend fetches data from and makes requests to, we are going to use a new pattern called **React Server Components (RSCs)**. This new pattern allows us to directly access the database from React components by executing certain React components (so-called server components) only on the server. Together with Server Actions (a way to call functions on the server from the client), this new pattern allows us to easily and quickly develop full-stack apps. In this chapter, we are going to learn what RSCs and Server Actions are, why they matter, what their advantages are, and how to implement them properly and securely.

In this chapter, we are going to cover the following main topics:

- What are RSCs?
- Adding a data layer to our Next.js app
- Using RSCs to fetch data from the database
- Using Server Actions to sign up, log in, and create new posts

Technical requirements

Before we start, please install all the requirements from [Chapter 1, Preparing For Full-Stack Development](#), and [Chapter 2, Getting to Know Node.js and MongoDB](#).

The versions listed in those chapters are the ones used in this book. While installing a newer version should not be an issue, please note that certain steps might work differently. If you are having an issue with the code and steps provided in this book, please try using the versions mentioned in *Chapters 1* and *2*.

You can find the code for this chapter on GitHub:

<https://github.com/PacktPublishing/Modern-Full-Stack-React-Projects/tree/main/ch17>.

The CiA video for this chapter can be found at:

<https://youtu.be/4hGZJRmZW6E>.

What are RSCs?

So far, we have been using the traditional React architecture, where all components are **client components**. We started with client-side rendering. However, there are some downsides to client-side rendering:

- The JavaScript client bundle must be downloaded from the server before the client can start rendering anything, delaying the **first contentful paint (FCP)** for the user.
- Data must be fetched from the server (after all JavaScript is downloaded and executed) to show anything meaningful, delaying the **first meaningful paint (FMP)** for the user.
- Most of the load is on the client, even for pages that are not interactive, which is especially problematic for clients with slow processors, such as low-end mobile devices or old laptops. It also uses more battery to load a heavy client-side rendered page.
- In certain cases, data is fetched sequentially (for example, loading posts first and then resolving the authors of each post), which is especially a problem for slow connections with high latency.

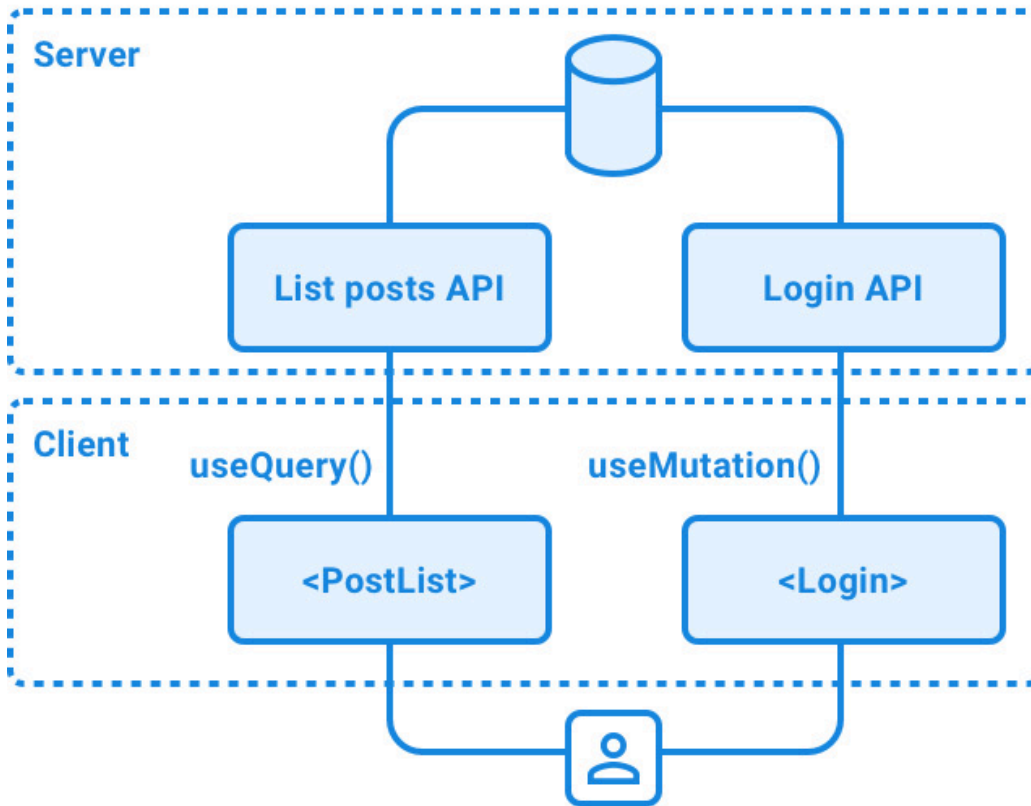
To solve these problems, **server-side rendering (SSR)** was introduced, but it still has a big downside: the initial page load can be slow due to everything being rendered on the server. This slowdown happens because of the following reasons:

- Data must be fetched from the server before any of it can be shown.
- The JavaScript client bundle must be downloaded from the server before the client can be hydrated with it. Hydration means that the page is ready to be interacted with by a user. To refresh your knowledge of how hydration works, check out [*Chapter 7*](#).
- Hydration has to be completed on the client before anything can be interacted with.

Even when a client component is pre-rendered on the server, its code will be bundled and sent to the client for hydration. This means that client components can run on both the server (for SSR) and the client, but they need to at least be able to run on the client.

In a traditional full-stack React architecture with only client components, if we needed to access the filesystem of the server or a database, we needed to write a separate backend using Node.js and expose an API (such as a REST API). Then, this API was queried in client components, for example, using TanStack Query. These queries can also be made on the server side (as we saw in [*Chapter 7, Improving the Load Time Using Server-Side Rendering*](#)), but they need to at least be executable on the client. This means we cannot directly access the filesystem or database from a React component, even if that code could run on the server; it would be bundled and sent to the client, where running it would not work (or expose internal information, such as credentials, to the database):

Without React Server Components



With React Server Components

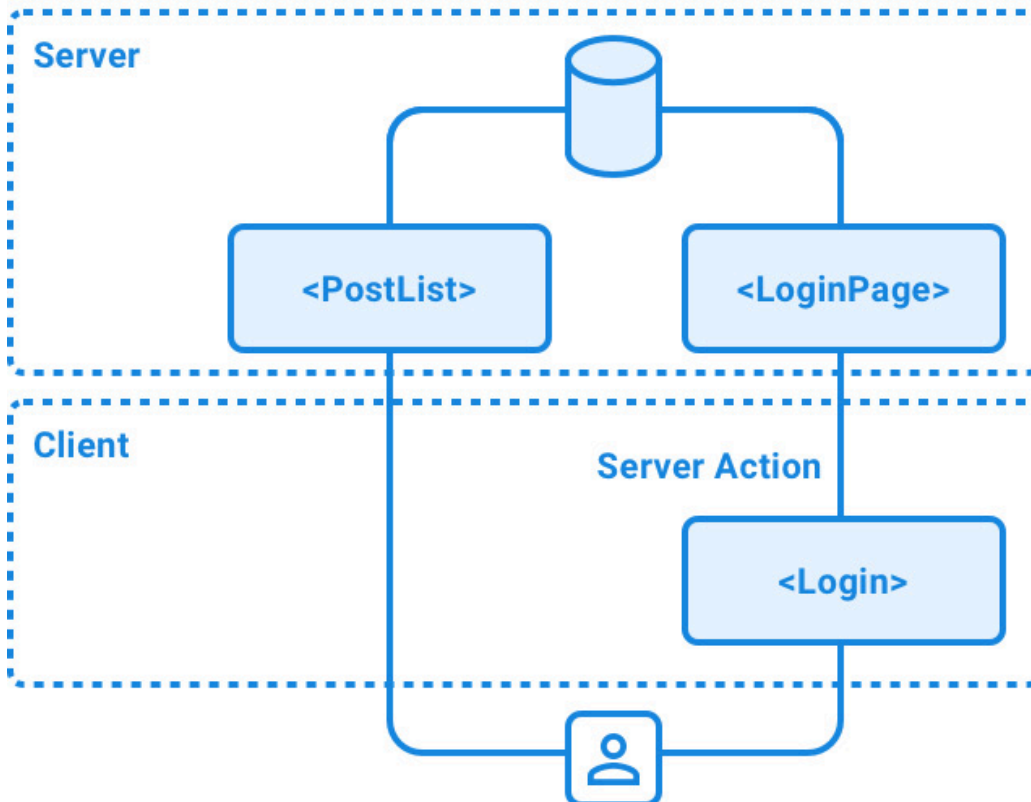


Figure 17.1 – The architecture of a full-stack app without and with RSCs

React 18 introduced a new feature called RSCs, which allows us to define components that will be solely executed on the server, with only the output sent to the client. Server components can, for example, fetch data from a database or the filesystem, and then render interactive client components, passing that data as props to them. This new feature allows for an architecture where we can more easily write a full-stack application using only React, without having to deal with the overhead of defining a REST API.

NOTE

It might still make sense to define REST APIs for certain apps, especially if the backend is developed by another team in a larger-scale project, or if it is consumed by other services and frontends.

RSCs solve the aforementioned issues with client-side rendering and SSR by allowing us to execute code exclusively on the server (no hydration needed on the client!) and selectively streaming components (so we don't have to wait for everything to pre-render before serving components to the client).

The following figure compares **client-side rendering (CSR)** with SSR and RSCs:

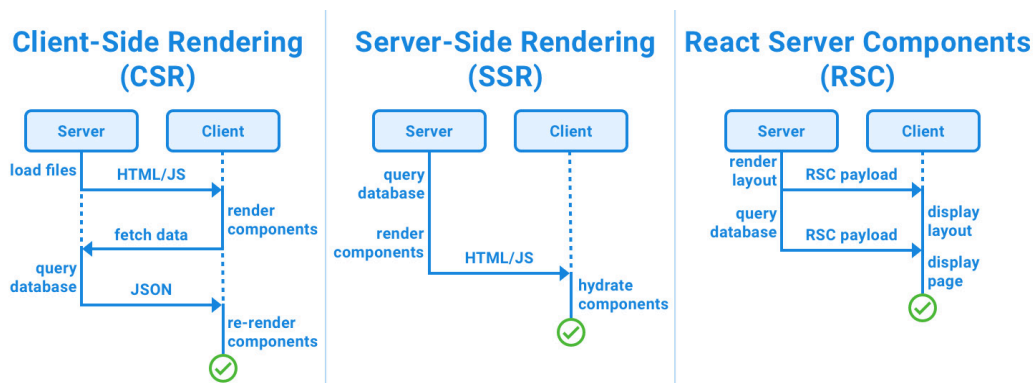


Figure 17.2 – Comparison between CSR, SSR and RSC

As you see, RSCs are not only faster overall (as a result of fewer roundtrips over the network), but they can also display the layout of an app immediately while waiting for the rest of the components to load.

Let's sum up the most important features of RSCs:

- They can run ahead of time and are excluded from the JavaScript bundle, reducing bundle size and improving performance.
- They can run either during build time (resulting in static HTML) or be executed on the fly when a request comes in. Interestingly, server components can also be exclusively executed during build time, resulting in a static HTML bundle. This can be useful for statically built CMS apps or personal blogs. RSCs also allow a mix, where the initial cache is primed with a static build, and then later revalidated through Server Actions or Webhooks. We are going to learn more about caching in [*Chapter 18, Advanced Next.js Concepts and Optimizations*](#).
- They can pass (serializable) data to client components. Additionally, client components can still be server-side rendered to further improve performance!
- Inside a server component, other server components can be passed as props to client components, allowing for composition patterns where server components are “slotted into” interactive client components. However, all components that are imported inside client components will be considered client components; they cannot be server components anymore.

In frameworks such as Next.js, by default, a React component is considered a server component. If we want to turn it into a client component, we need to write the **"use client"** directive at the beginning of a file. We need to do this to make it possible to add interactivity (event listeners) or use state/life cycle effects and browser-only APIs.

NOTE

*The **"use client"** directive defines a network boundary between server and client components. All data sent from a server component to a client component will be serialized and sent over the network. When using the **"use client"** directive in a file, all other modules that are imported into it, including child components, are considered to be part of the client bundle.*

The following figure provides an overview of when to use a server component or a client component:

What do you need to do?	Server Component	Client Component
Fetch data	✓	✗
Access backend resources (directly)	✓	✗
Keep sensitive information on the server (access tokens, API keys, etc)	✓	✗
Keep large dependencies on the server (reduces client-side JavaScript)	✓	✗
Add interactivity and event listeners (onClick, onChange, etc)	✗	✓
Use state and lifecycle effects (useState, useReducer, useEffect, etc)	✗	✓
Use browser-only APIs	✗	✓
Use custom hooks that depend on state, effects, or browser-only APIs	✗	✓
Use React class components	✗	✓

Figure 17.3 – Overview of when to use server components and client components

In general, RSCs are an optimization over client components. You could simply write `"use client"` at the top of every file and be done with it, but you would be leaving all the advantages of RSCs behind! So, try to use server components whenever possible, but do not hesitate to fall back to defining something as a client component if it turns out to be too complicated to split it up into server-side and client-side parts. It can always be optimized later.

This new way of writing full-stack React applications can be hard to grasp in theory, so feel free to come back to this section again at the end of this chapter. For now, we'll move on and implement RSCs in our Next.js app as this will help us understand how the new concepts work in practice. First, we'll start by adding a data layer to our Next.js app, which will allow us to access the database from RSCs later on.

Adding a data layer to our Next.js app

In the traditional backend structure, we had the database layer, the services layer, and the routes layer. In a modern full-stack Next.js app, we don't need the routes layer of our backend because we can directly interface with it in RSCs. So, we only need to have the database layer and a data layer to provide functions that access the database. Theoretically, we could directly access the database in RSCs, but it is best practice to have specific functions that access it in certain ways. Defining such functions allows us to clearly define what data is accessible (and thus avoid accidentally leaking too much information). They are also more reusable and make it easier to unit-test and find potential vulnerabilities (for example, via a penetration test) in the data layer.

To recap, there are three main data-handling approaches:

- **HTTP APIs:** We used these in previous chapters to implement our blog app. These can be useful when separate teams are working on the backend and frontend. Due to this, this approach is recommended for existing large projects and organizations.
- **Data access layer:** This is the pattern we are going to use in this section. It is recommended for new projects that make use of the RSC architecture as it makes it easier to implement full-stack projects by separating concerns of dealing with data (and all the security challenges associated with that) and the user interface (displaying the data in React components). Dealing with each problem on its own is easier to solve and less error-prone than handling the complexity of both at once.
- **Component-level data access:** This is a pattern where the database is queried directly in RSCs. This approach can be useful for rapid prototyping and learning. However, it should not be used in a production app due to scalability issues and the potential introduction of security problems.

It is not recommended to mix these approaches, so it's better to pick one and stick to it. In our case, we are going with the “data access layer” ap-

proach as it is the safest approach for a modern RSC architecture.

Setting up the database connection

Let's start by setting up the necessary packages and initializing a database connection:

1. Copy the existing **ch16** folder to a new **ch17** folder, as follows:

```
$ cp -R ch16 ch17
```

2. Open the **ch17** folder in VS Code and open a Terminal.
3. We are going to use a package called **server-only** to make sure code from the database and data layer are only executed on the server-side, and not accidentally imported on the client. Install it, as follows:

```
$ npm install server-only@0.0.1
```

4. We are also going to need the **mongoose** package to connect to the database and create database schemas and models. Run the following command to install it:

```
$ npm install mongoose@8.0.2
```

5. Create a new **src/db/** folder.
6. Inside this folder, create a new **src/db/init.js** file, in which we first import the **server-only** package to make sure the code is only executed on the server:

```
import 'server-only'
```

7. Next, import **mongoose**:

```
import mongoose from 'mongoose'
```

8. Define and export an **async** function to initialize the database:

```
export async function initDatabase() {  
  const connection = await mongoose.connect(process.env.DATABASE_URL)
```

```
    return connection
  }
```

9. Now, we need to define **DATABASE_URL** in a **.env** file. So, create a new **.env** file in the root of the project and add the following line:

```
DATABASE_URL=mongodb://localhost:27017/blog
```

Now that the database connection has been set up, we can move on to creating the database models.

Creating the database models

Now, we are going to create database models for posts and users. These are going to be very similar to the ones we created for our blog app in previous chapters. Follow these steps to start creating the database models:

1. Create a new **src/db/models/** folder.
2. Inside it, create a new **src/db/models/user.js** file, where we first import the **server-only** and **mongoose** packages:

```
import 'server-only'
import mongoose, { Schema } from 'mongoose'
```

3. Define **userSchema**, which consists of a unique required **username** and a required **password**:

```
const userSchema = new Schema({
  username: { type: String, required: true, unique: true },
  password: { type: String, required: true },
})
```

4. We create the Mongoose model if it has not been created yet:

```
export const User = mongoose.models.user ?? mongoose.model('user', userSchema)
```

NOTE

*Returning the model if it already exists and only creating a new one if it does not is necessary to avoid an **OverwriteModelError** issue, which happens when the model is imported (and thus redefined) multiple times.*

5. Create a new **src/db/models/post.js** file, where we first import the **server-only** and **mongoose** packages:

```
import 'server-only'
import mongoose, { Schema } from 'mongoose'
```

6. Define **postSchema**, which consists of a required **title** and **author** (referencing the **user** model), and optional **contents**:

```
const postSchema = new Schema(
  {
    title: { type: String, required: true },
    author: { type: Schema.Types.ObjectId, ref: 'user', required: true },
    contents: String,
  },
  { timestamps: true },
)
```

7. We create the Mongoose model if it has not been created yet:

```
export const Post = mongoose.models.post ?? mongoose.model('post', postSchema)
```

8. Create a new **src/db/models/index.js** file and re-export the models:

```
import 'server-only'
export * from './user'
export * from './post'
```

We re-export the models from this folder to ensure that we can, for example, load a post and resolve the **author** by querying the corresponding user. This would require defining the **user** model, although it is not directly used. To avoid issues like these, we simply load models from a file that defines all models upon importing them.

After defining the database models, we can define the data layer functions, which will provide various ways to access the database.

Defining data layer functions

Now that we have a database connection and schemas, let's start defining data layer functions that access the database.

Defining the posts data layer

We'll start by defining the data layer for posts. This allows us to access all the relevant functions for dealing with posts in our app:

1. Create a new **src/data/** folder.
2. Inside it, create a new **src/data/posts.js** file, where we import the **server-only** package and the **Post** model:

```
import 'server-only'
import { Post } from '@db/models'
```

3. Define a **createPost** function that takes a **userId**, **title**, and **contents** and creates a new post:

```
export async function createPost(userId, { title, contents }) {
  const post = new Post({ author: userId, title, contents })
  return await post.save()
}
```

4. Next, define a **listAllPosts** function, which first gets all posts from the database sorted by creation date descending (showing the newest posts first):

```
export async function listAllPosts() {
  return await Post.find({})
    .sort({ createdAt: 'descending' })
}
```

5. Then, we must **populate** the **author** field by resolving the **user** model and getting the **username** value from it:

```
.populate('author', 'username')
```

In Mongoose, the **populate** function works like a **JOIN** statement in SQL: it takes the ID stored in the **author** field and then checks which

model the ID references by looking at the **post** schema. In the **post** schema, we defined that the **author** field references the **user** schema, so Mongoose will query the **user** model for the given ID and return a user object. By providing the second argument, we specify that we only want to get the **username** value from the **user** object (the ID will always be returned anyway). This is done to avoid leaking internal information, such as the (hashed) password of a user.

6. After populating the post objects, we use **.lean()** to turn it into a plain, serializable JavaScript object:

```
.lean()  
}
```

Having a serializable object is necessary to be able to pass the data from an RSC to a regular client-side component later since all data passed to the client needs to cross the network boundary, and thus needs to be serializable.

7. Lastly, we must define a **getPostById** function, which finds a single post by ID, populates the **author** field, and turns the result into a plain JavaScript object by using **lean()**:

```
export async function getPostById(postId) {  
  return await Post.findById(postId)  
    .populate('author', 'username')  
    .lean()  
}
```

Defining the data layer for users

We are now going to define the data layer for users. This will involve creating a JWT for authentication. Again, a lot of the code will be very similar to what we previously implemented for our blog app. Follow these steps to start defining the data layer for users:

1. Install **bcrypt** (for hashing the user password) and **jsonwebtoken** (for handling JWTs):

```
$ npm install bcrypt@5.1.1 jsonwebtoken@9.0.2
```

2. Create a new **src/data/users.js** file, where we import **server-only**, **bcrypt**, **jwt**, and the **User** model:

```
import 'server-only'
import bcrypt from 'bcrypt'
import jwt from 'jsonwebtoken'
import { User } from '@db/models'
```

3. Define a **createUser** function, where we hash the given password and then create a new instance of the **User** model and save it:

```
export async function createUser({ username, password }) {
  const hashedPassword = await bcrypt.hash(password, 10)
  const user = new User({ username, password: hashedPassword })
  return await user.save()
}
```

4. Next, define a **loginUser** function, which first tries to find a user with the given username and throws an error if no user is found:

```
export async function loginUser({ username, password }) {
  const user = await User.findOne({ username })
  if (!user) {
    throw new Error('invalid username!')
  }
}
```

NOTE

Depending on your security requirements, you might want to consider not telling a potential attacker that a username exists and instead return a generic message such as “Invalid username or password.” However, in our case, the usernames are assumed to be public information because each user is an author on the blog and their usernames are published with their articles.

5. Then, use **bcrypt** to compare the provided password against the hashed password from the database and throw an error if the password is invalid:

```
const isPasswordCorrect = await bcrypt.compare(password, user.password)
if (!isPasswordCorrect) {
  throw new Error('invalid password!')
}
```

6. Lastly, generate, sign, and return a JWT:

```
const token = jwt.sign({ sub: user._id }, process.env.JWT_SECRET, {
  expiresIn: '24h',
})
return token
}
```

7. Now, we are going to define a function to get the user information (for now, we're only going to get the username, but this could be extended later) from a user ID. If the user ID does not exist, we throw an error:

```
export async function getUserInfoById(userId) {
  const user = await User.findById(userId)
  if (!user) throw new Error('user not found!')
  return { username: user.username }
}
```

8. Next, define a function to get the user ID from a token, making sure to verify the token signature in addition to decoding the JWT, by using **jwt.verify**:

```
export function getUserIdByToken(token) {
  if (!token) return null
  const decodedToken = jwt.verify(token, process.env.JWT_SECRET)
  return decodedToken.sub
}
```

9. Finally, define a function to get the user information from a token by combining the **getUserIdByToken** and **getUserInfoById** functions:

```
export async function getUserInfoByToken(token) {
  const userId = getUserIdByToken(token)
  if (!userId) return null
  const user = await getUserInfoById(userId)
}
```



```
    return user
  }
```

10. We still need to define the `JWT_SECRET` environment variable for our code to work. Edit `.env` and add it, as follows:

```
JWT_SECRET=replace-with-random-secret
```

NOTE

This is a very basic implementation of authentication with Next.js. For a large-scale project, it is recommended to look into a fully-fledged authentication solution, such as Auth.js (formerly next-auth), Auth0, or Supabase. Check out the Next.js docs for more information on authentication with Next.js: <https://nextjs.org/docs/app/building-your-application/authentication>.

Now that we have a data layer to access the database, we can start implementing RSCs and Server Actions, which are going to call functions from the data layer to access information from the database and render React components that display it, turning our static blog app into a fully functional blog.

Using RSCs to fetch data from the database

As we have learned, when using Next.js, React components are considered to be Server Components by default, so all page components are already executed and rendered on the server. Only if we need to use client-only functions, such as hooks or input fields, do we need to turn our components into a client component by using the `"use client"` directive. For all components that do not require user interaction, we can simply keep them as server components, and they will only be statically rendered and served as HTML (encoded in the RSC payload) and not hydrated on the client. To the client (the browser), it will seem as if these React components don't even exist as the browser will only see static HTML code. This pattern greatly improves the performance of our web application as the

client doesn't need to load JavaScript to render such components. It also reduces the bundle size because less JavaScript code is needed to load our web application.

Now, let's implement RSCs to fetch data from the database.

Fetching a list of posts

We'll start by implementing the **HomePage**, where we fetch and render a list of posts:

1. Edit **src/app/page.js** and import the **initDatabase** and **listAllPosts** functions:

```
import { initDatabase } from '@db/init'
import { listAllPosts } from '@data/posts'
```

2. Turn the **HomePage** component into an **async** function, which allows us to wait until the data is fetched before rendering the component:

```
export default async function HomePage() {
```

3. *Replace* the sample **posts** array with the following code:

```
await initDatabase()
const posts = await listAllPosts()
```

Fetching a single post

Now that we can view a list of posts, let's continue by implementing the process of fetching a single post for **ViewPostPage**. Follow these steps to get started:

1. Edit **src/app/posts/[id]/page.js** and import the **notFound**, **getPostById**, and **initDatabase** functions:

```
import { notFound } from 'next/navigation'
```

```
import { getPostById } from '@data/posts'
import { initDatabase } from '@db/init'
```

2. Turn the page component into an **async** function:

```
export default async function ViewPostPage({ params }) {
```

3. Replace the sample **post** object with calls to **initDatabase** and **getPostById**:

```
await initDatabase()
const post = await getPostById(params.id)
```

If no post was found, we call the **notFound** function, which throws a **NEXT_NOT_FOUND** error and terminates the rendering of the route segment:

```
if (!post) notFound()
```

Now, we need to create a **not-found.js** file to catch the error and render a different component instead.

4. Create a new **src/app/posts/[id]/not-found.js** file, where we render a “Post not found!” message, as follows:

```
export default function ViewPostError() {
  return <strong>Post not found!</strong>
}
```

TIP

*We can also add an **app/not-found.js** file to handle unmatched URLs for the whole application. If users access a path that is not defined by the app, the component defined in that file will be rendered instead.*

5. Additionally, we can create an error component that will be rendered for any errors, such as not being able to connect to the database. Create a new **src/app/posts/[id]/error.js** file, where we render an “Error while loading the post!” message, as follows:

```
'use client'
export default function ViewPostError() {
```

```
    return <strong>Error while loading the post!</strong>
  }
}
```

Error pages need to be client components, so we added a **'use client'** directive.

INFO

*The reason why error pages need to be client components is that they use the React **ErrorBoundary** feature, which is implemented as class components (using **componentDidCatch**). React class components cannot be server components, so we need to make the error page a client component.*

6. We still need to make a small adjustment to the **Post** component because the **_id** is now actually not a string anymore; instead, it's an **ObjectId** object. Edit **src/components/Post.jsx** and change the type, as follows:

```
Post.propTypes = {
  _id: PropTypes.object.isRequired,
```

7. Make sure Docker and the MongoDB container are running properly!
8. Run the dev server, as follows:

```
$ npm run dev
```

9. Go to **http://localhost:3000** and click on one of the posts in the list; you will see that the post loads successfully. If a post does not exist (for example, if you change a single digit in the ID), the “Post not found!” message will be shown. If there was any other error (for example, an invalid ID), the “Error while loading the post!” message will be shown:

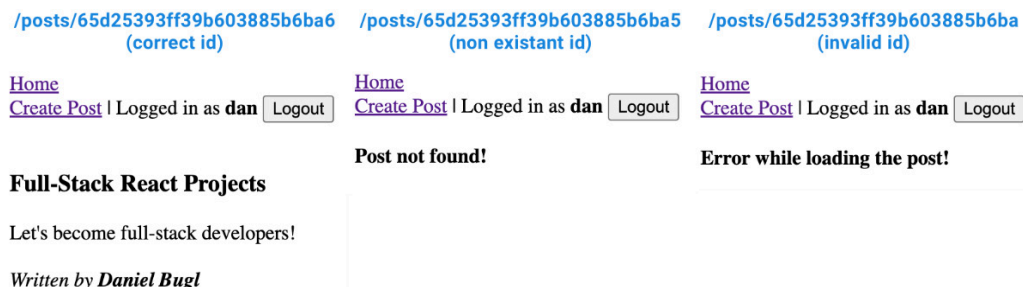


Figure 17.4 – Showing a post and the not found/error components

NOTE

If there are no posts in your database yet, either create a new post by using the blog app from earlier chapters or wait until we implement the create post functionality using Next.js at the end of this chapter.

After implementing RSCs for fetching posts, our blog app is now connected to the database. However, all it can do right now is show posts; there is no way for the user to interact with the app yet. Let's move on to making our blog app interactive by adding Server Actions to it.

Using Server Actions to sign up, log in, and create new posts

So far, we have only been fetching data from the database on the server and sending it to the client, but for user interactivity, we need to be able to send data back from the client to the server. To be able to do this, React introduced a pattern called Server Actions.

Server Actions are functions that are executed on the server side but can be triggered from the client side, for example, through a form submission. This can be done even with JavaScript disabled on the client, which will then submit the form using regular form submission. When JavaScript is enabled, the form will be progressively enhanced and not require a full refresh to submit. These functions are defined by tagging regular JavaScript functions with the **"use server"** directive, and then either importing them into a client component or passing them to a client component via props. While regular JavaScript functions cannot be passed to client components (because they aren't serializable), Server Actions can be.

NOTE

*You can define a whole file to be full of Server Actions by adding the **"use server"** directive at the beginning of a file. This will tell the bundler that all functions in this file are Server Actions; it does not define components inside it as server components (to enforce something to be executed on the*

*server, use the **server-only** package instead, as explained). You can then import functions from such a file in client components.*

In client components, we can make use of the **useFormState** hook, which has a similar signature to **useState** but allows us to execute server actions (on the server) and get back the result on the client. The **useFormState** hook's signature looks as follows:

```
const [state, formAction] = useFormState(fn, initialState)
```

NOTE

*In the React 19 release, **useFormState** hook will be renamed to **useActionState**. See <https://react.dev/reference/react/useActionState> for more information.*

As we can see, we pass in a function (Server Action) and an initial state. The hook then returns the current state and a **formAction** function. The state is initially set to the initial state, and updated to the result of the Server Action after the **formAction** function is called. On the server side, the Server Action signature looks as follows:

```
function exampleServerAction(previousState, formData) {  
  "use server"  
  // ...do something...  
}
```

As we can see, the Server Action function accepts **previousState** (which will initially be set to **initialState** from the client) and a **formData** object (which is a regular **formData** object from the XMLHttpRequest API web standard). The **formData** object contains all information submitted in form fields. This allows us to easily submit forms to perform an action on the server and return the result to the client.

Now, let's start using Server Actions to implement the signup page in our blog app.

Implementing the signup page

The first action a user needs to take to interact with the blog app is signing up, so let's start by implementing this feature. Follow these steps to get started:

1. We start by implementing the client component. Edit `src/components/Signup.jsx` and mark it as a client component, then import the `useFormState` hook and `PropTypes`:

```
'use client'
import { useFormState } from 'react-dom'
import PropTypes from 'prop-types'
```

2. The **Signup** component now needs to accept a **signupAction**, which we are going to define on the server side later:

```
export function Signup({ signupAction }) {
```

3. Define a **useFormState** hook, which takes a Server Action and an initial state (in our case, an empty object), and returns the current state and an action:

```
const [state, formAction] = useFormState(signupAction, {})
```

4. Now, we can add **action** to the `<form>` tag, as follows:

```
return (
  <form action={formAction}>
```

The action will automatically be called when the form is submitted. Alternatively, Server Actions can be called by executing them like a regular async function – for example, by calling `await formAction()` inside an `onClick` handler function.

5. Additionally, we can show an error message below the “Sign Up” button if we get a **state.error** message back from the server:

```
<input type='submit' value='Sign Up' />
{state.error ? <strong> Error signing up: {state.error}</strong> : null}
</form>
```

```
)  
}
```

6. Let's not forget to define **propTypes** for the **Signup** component. **signupAction** is a function:

```
Signup.propTypes = {  
  signupAction: PropTypes.func.isRequired,  
}
```

7. Now, we can start implementing the actual server action. Edit **src/app/signup/page.js** and import the **redirect** function from **next/navigation** (to navigate to the login page after successfully signing up), as well as the **createUser** and **initDatabase** functions:

```
import { redirect } from 'next/navigation'  
import { createUser } from '@data/users'  
import { initDatabase } from '@db/init'  
import { Signup } from '@components/Signup'
```

8. Then, outside of the **SignupPage** component, define a new **async** function that accepts the previous state (in our case, this is the empty object we defined as the initial state, so we can ignore it) and a **formData** object:

```
async function signupAction(prevState, formData) {
```

9. We need to tag the function with the **'use server'** directive to turn it into a Server Action:

```
'use server'
```

10. Then, we can initialize the database and attempt to create a user:

```
try {  
  await initDatabase()  
  await createUser({  
    username: formData.get('username'),  
    password: formData.get('password'),  
  })  
}
```


As you can see, Server Actions build upon existing web APIs and use the **FormData** API for form submission. We can simply call **.get()** with the **name** of the input field and it will contain the value provided in the respective input field.

11. If there is an error, we return the error message (which will then be shown in the **Signup** client component):

```
    } catch (err) {  
      return { error: err.message }  
    }
```

12. Otherwise, if everything went well, we redirect to the login page:

```
    redirect('/login')  
  }
```

13. After defining the Server Action, we can pass it to the **Signup** component, as follows:

```
export default function SignupPage() {  
  return <Signup signupAction={signupAction} />  
}
```

Alternatively, the client component could directly import the **signupAction** function from a file. So long as the function has the **'use server'** directive, it will be executed on the server. In this case, we only need the function on this specific page, so it makes more sense to define it on the page and pass it to the component.

14. Run the dev server, as follows:

```
$ npm run dev
```

15. Go to **http://localhost:3000/signup** and try entering a username and password. It should work successfully and redirect you to the login screen (the change is subtle, but the submit button changes from **Sign Up** to **Log In**).
16. Go to **http://localhost:3000/signup** again and try entering the same username. You will get the following error:

Username:

Password:

Error signing up: E11000 duplicate key error collection: ch3.users index: username_1 dup key: { username: "test42" }

Figure 17.5 – An error is shown when the username already exists

Of course, this error message is not very user-friendly, so we could do some work to improve the error messages here. But for now, this is sufficient as an example to show how Server Actions work.

As you can see, RSCs and Server Actions make implementing features that interface with the database straightforward. As an additional bonus, all Server Actions that are submitted via `<form>` even work with JavaScript disabled – try it out by repeating *Steps 15* and *16* with JavaScript disabled!

Implementing the login page and JWT handling

Now that users can sign up, we need a way for them to log in. This also means that we will need to implement functionality to create and store JWT. Now that we have more control over the server-client interaction with Next.js, we can store the JWT in a cookie instead of in memory. This means that the user session will persist even when they refresh the page.

Let's start implementing the login page and JWT handling:

1. We'll start by implementing the client component. Edit `src/components/Login.jsx` and turn it into a client component:

```
'use client'
```

2. Then, import the `useFormState` hook and `PropTypes`:

```
import { useFormState } from 'react-dom'
import PropTypes from 'prop-types'
```

3. Accept `loginAction` as props. We are going to use this to define the `useFormState` hook:

```
export function Login({ loginAction }) {
```

```
const [state, formAction] = useFormState(loginAction, {})
```

4. Pass **formAction**, which was returned from the hook, to the **<form>** element:

```
return (
  <form action={formAction}>
```

5. Now, we can display potential errors at the end of the component:

```
    <input type='submit' value='Log In' />
    {state.error ? <strong> Error logging in: {state.error}</strong> : null}
  </form>
)
}
```

6. Lastly, define **propTypes**, as follows:

```
Login.propTypes = {
  loginAction: PropTypes.func.isRequired,
}
```

7. Now, we can create the **loginAction** Server Action. Edit **src/app/login/page.js** and import the **cookies** and **redirect** functions from Next.js, as well as the **loginUser** and **initDatabase** functions from our data layer:

```
import { cookies } from 'next/headers'
import { redirect } from 'next/navigation'
import { loginUser } from '@data/users'
import { initDatabase } from '@db/init'
import { Login } from '@components/Login'
```

8. Define a new **loginAction** outside of the **LoginPage** component, in which we attempt to log in with the given username and password:

```
async function loginAction(prevState, formData) {
  'use server'
  let token
  try {
    await initDatabase()
    token = await loginUser({
```

```
    username: formData.get('username'),  
    password: formData.get('password'),  
  })
```

9. If this fails, we return the error message:

```
  } catch (err) {  
    return { error: err.message }  
  }
```

10. Otherwise, we set an **AUTH_TOKEN** cookie with an expiry of 24 hours (the same expiry time as the JWT we created), and make it **secure** and **httpOnly**:

```
cookies().set({  
  name: 'AUTH_TOKEN',  
  value: token,  
  path: '/',  
  maxAge: 60 * 60 * 24,  
  secure: true,  
  httpOnly: true,  
})
```

NOTE

*The **httpOnly** attribute makes sure cookies cannot be accessed by client JavaScript, reducing the possibility of cross-site scripting attacks in our app. The **secure** attribute ensures that the cookie is set on the HTTPS version of the website. To improve the development experience, this doesn't apply to localhost.*

11. After setting the cookie, we redirect to the home page:

```
  redirect('/')  
}
```

12. Finally, we pass the **loginAction** to the **Login** component:

```
export default function LoginPage() {
```

```
return <Login loginAction={loginAction} />
}
```

- Go to **http://localhost:3000/login** and try entering a username that doesn't exist; you will get an error. Then, try entering the same username and password that you used to sign up earlier. It should work successfully and redirect you to the home page.

Checking if the user is logged in

You may have noticed that after the user logs in, the navigation bar doesn't change. We still have to check if the user is logged in and then adjust the navigation bar accordingly. Let's do that now:

- Edit **src/app/layout.js** and import the **cookies** function from Next.js and the **getUserInfoByToken** function from our data layer:

```
import { cookies } from 'next/headers'
import { getUserInfoByToken } from '@data/users'
import { Navigation } from '@components/Navigation'
```

- Turn **RootLayout** into an **async** function:

```
export default async function RootLayout({ children }) {
```

- Get the **AUTH_TOKEN** cookie and pass its value to the **getUserInfoByToken** function to get the **user** object, *replacing* the sample **user** object we defined earlier:

```
const token = cookies().get('AUTH_TOKEN')
const user = await getUserInfoByToken(token?.value)
```

- If you still have the home page open from earlier, it should hot reload automatically and show your username and the logout button.

We are already passing **user?.username** to the **Navigation** component, so that's all there is to it!

Implementing logout

Now that we can show a different navigation bar when the user is logged in, we can finally see the logout button. However, it doesn't work yet. We'll implement the logout button now:

1. Edit **src/app/layout.js** and define a **logoutAction** Server Action outside of the **RootLayout** component:

```
async function logoutAction() {  
  'use server'
```

2. Inside this action, we simply delete the **AUTH_TOKEN** cookie:

```
  cookies().delete('AUTH_TOKEN')  
}
```

3. Pass **logoutAction** to the **Navigation** component, as follows:

```
<Navigation  
  username={user?.username}  
  logoutAction={logoutAction}  
/>
```

4. Edit **src/components/Navigation.jsx** and add **logoutAction** to **UserBar** and the logout form:

```
export function UserBar({ username, logoutAction }) {  
  return (  
    <form action={logoutAction}>
```

5. Add the action to **propTypes** of the **UserBar** component, as follows:

```
UserBar.propTypes = {  
  username: PropTypes.string.isRequired,  
  logoutAction: PropTypes.func.isRequired,  
}
```

6. Then, add **logoutAction** as props to the **Navigation** component and pass it down to the **UserBar** component:

```
export function Navigation({ username, logoutAction }) {  
  return (  
    <UserBar username={username} logoutAction={logoutAction} />
```

```

    <>
      <Link href="/">Home</Link>
      {username ? (
        <UserBar
          username={username}
          logoutAction={logoutAction}
        />
      ) : (
        <LoginSignupLinks />
      )}
    </>
  )
}

```

7. Finally, change **propTypes** of the **Navigation** component, as follows:

```

Navigation.propTypes = {
  username: PropTypes.string,
  logoutAction: PropTypes.func.isRequired,
}

```

8. Click the **Logout** button to see the navigation bar change back to show the **Log In** and **Sign Up** links.

Now, our users can finally log in and log out again successfully. Let's move on to implementing post creation.

Implementing post creation

The last feature missing in our blog app is post creation. We can use Server Actions and a JWT to authenticate the user and allow them to create a post. Follow these steps to implement post creation:

1. This time, we start by implementing the Server Action. Edit **src/app/create/page.js** and import the **cookies**, **redirect**, **createPost**, **getUserIdByToken**, and **initDatabase** functions:

```

import { cookies } from 'next/headers'
import { redirect } from 'next/navigation'
import { createPost } from '@data/posts'
import { getUserIdByToken } from '@data/users'

```

```
import { initDatabase } from '@db/init'
import { CreatePost } from '@components/CreatePost'
```

2. Inside the **CreatePostPage** component, get the token from the cookie:

```
export default function CreatePostPage() {
  const token = cookies().get('AUTH_TOKEN')
```

3. Still inside the **CreatePostPage** component, define a Server Action:

```
async function createPostAction(formData) {
  'use server'
```

We won't be using the **useFormState** hook this time because we don't need to handle the state or result of the action on the client side. So, the Server Action does not have the (**prevState**, **formData**) signature and instead has a (**formData**) signature.

4. Inside the Server Action, we get the **userId** value from the token, then initialize the database connection and create a new post:

```
const userId = getUserIdByToken(token?.value)
await initDatabase()
const post = await createPost(userId, {
  title: formData.get('title'),
  contents: formData.get('contents'),
})
```

5. Lastly, we redirect to the **ViewPost** page of the newly created post:

```
redirect(`/posts/${post._id}`)
}
```

6. If the user isn't logged in, we can now show an error message:

```
if (!token?.value) {
  return <strong>You need to be logged in to create posts!</strong>
}
```

7. Otherwise, we render the **CreatePost** component, passing **createPostAction** to it:

```
return <CreatePost createPostAction={createPostAction} />
```



```
}
```

8. Now, we can adjust the **CreatePost** component. We *don't* need to turn it into a client component this time because we won't be using the **useFormState** hook. Edit **src/components/CreatePost.jsx** and import **PropTypes**:

```
import PropTypes from 'prop-types'
```

9. Then, add the **createPostAction** as props and pass it to the form element:

```
export function CreatePost({ createPostAction }) {  
  return (  
    <form action={createPostAction}>
```

10. Finally, define **propTypes**, as follows:

```
CreatePost.propTypes = {  
  createPostAction: PropTypes.func.isRequired,  
}
```

11. Go to **http://localhost:3000**, log in again, and then click the **Create Post** link. Enter a title and some contents and click the **Create** button; you should get redirected to the **ViewPost** page of the newly created blog post!

Summary

In this chapter, we learned about RSCs, why they were introduced, what their advantages are, and how they fit into our full-stack architecture. Then, we learned how to safely implement RSCs by introducing a data layer into our app. Afterward, we fetched data from our database and rendered components using RSCs. Finally, we learned about Server Actions and added interactive features to our blog app. Now, our blog app is fully functional again!

In the next chapter, [***Chapter 18**, Advanced Next.js Concepts and Optimizations*](#), we are going to dive deep into how Next.js works and how

we can further optimize our app when using it. We are going to learn about caching, image and font optimizations, and how to define metadata for SEO optimization.